

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Sementara
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikroprosesor

Vinsen Dwi Putra - 5024241094

2026

1 Pendahuluan

1.1 Latar Belakang

Perkembangan teknologi sistem tertanam (*embedded system*) modern menuntut integrasi perangkat keras dan perangkat lunak yang andal untuk berbagai kebutuhan instrumentasi serta otomasi. Dalam praktiknya, pemilihan platform mikrokontroler sangat bergantung pada kebutuhan komputasi dan arsitekturnya, seperti keluarga ARM Cortex-M pada STM32 Blackpill yang berfokus pada eksekusi deterministik, maupun ESP32 berbasis prosesor Xtensa yang terintegrasi secara bawaan dengan komunikasi nirkabel. Keberagaman arsitektur ini memerlukan lingkungan pengembangan terpadu seperti PlatformIO yang mampu menjembatani perbedaan rantai perkakas (*toolchain*) serta mendukung beragam kerangka kerja, mulai dari abstraksi tingkat tinggi Arduino hingga kerangka kerja tingkat lanjut seperti ESP-IDF.

Selain penguasaan arsitektur dan lingkungan pemrograman, interaksi fisik dengan lingkungan luar melalui pin *General Purpose Input/Output* (GPIO) merupakan fondasi utama perancangan sistem tertanam. Mode pembacaan masukan digital pada komponen sakelar rentan terhadap gangguan status melayang (*floating*) serta fenomena pentalan mekanis (*bouncing*) yang memerlukan penerapan resistor *pull-up*/*pull-down* dan algoritma *debouncing*. Di samping masukan digital biner, sistem tertanam juga perlu mengolah besaran fisis kontinu melalui modul periferal *Analog-to-Digital Converter* (ADC) seperti pada pembacaan sensor cahaya LDR. Oleh karena itu, praktikum ini dilaksanakan untuk memberikan pemahaman menyeluruh mengenai arsitektur perangkat, konfigurasi berbagai mode akses GPIO, serta teknik akuisisi data yang stabil.

1.2 Dasar Teori

1.2.1 PlatformIO dan Visual Studio Code

PlatformIO merupakan ekosistem terpadu yang dirancang untuk memfasilitasi pengembangan perangkat lunak pada sistem tertanam secara lintas platform. Melalui integrasi dengan Visual Studio Code, PlatformIO menyediakan manajemen rantai perkakas (*toolchain*), pustaka dependensi, hingga protokol pengunggahan dan pemantauan serial dalam satu antarmuka yang terpusat. Keunggulan utamanya terletak pada fleksibilitas penyesuaian berbagai target mikrokontroler dan *framework* pemrograman yang berbeda hanya melalui pengaturan deklaratif pada berkas `platformio.ini`.

Dalam struktur proyek PlatformIO, berkas sumber utama diorganisasikan pada direktori `src/`, berkas *header* pada direktori `include/`, serta pustaka tambahan pada direktori `lib/`. Pendekatan ini memungkinkan pengembang beralih dari satu arsitektur perangkat keras ke arsitektur lain secara cepat tanpa harus mengubah konfigurasi editor dasar. Efisiensi tersebut mempermudah pengujian modul pada platform seperti STM32 maupun ESP32 secara konsisten.

1.2.2 Arsitektur STM32 Blackpill dan Framework Arduino

Board STM32 Blackpill ditenagai oleh mikrokontroler STM32F401CCU6 berbasis arsitektur 32-bit ARM Cortex-M4 dengan frekuensi komputasi mencapai 84 MHz. Papan pengembang ini dilengkapi dengan unit komputasi titik kambang (*Floating Point Unit*), memori Flash sebesar 256 KB, serta SRAM 64 KB. Karena tidak memiliki sirkuit pemrogram bawaan (*on-board programmer*), pengunggahan kode maupun *debugging* memerlukan perangkat eksternal seperti ST-Link V2 yang terhubung

melalui jalur antarmuka *Serial Wire Debug* (SWD).

Penggunaan *framework* Arduino melalui dukungan STM32duino memberikan kemudahan karena menyediakan lapisan abstraksi perangkat lunak tingkat tinggi yang umum digunakan, seperti fungsi `pinMode()` dan `digitalWrite()`. Keistimewaan implementasi ini terlihat pada kemampuan pengalamatan pin secara langsung sesuai label fisik mikrokontroler, misalnya `PC13` untuk LED indikator bawaan dan `PA0` untuk tombol pengguna. Hal ini mempermudah transisi pemahaman logika kontrol digital sebelum memasuki manipulasi register perangkat keras tingkat rendah.

1.2.3 Arsitektur ESP32 dan Framework ESP-IDF

ESP32 merupakan modul mikrokontroler bersistem monolitik (*System on Chip*) berbasis prosesor Xtensa Dual-Core 32-bit LX6 yang telah mengintegrasikan modul komunikasi nirkabel Wi-Fi serta Bluetooth. Dilengkapi antarmuka pengonversi USB-ke-Serial pada papan pengembangnya, ESP32 memungkinkan proses pengunggahan kode dan komunikasi data langsung melalui kabel USB tanpa memerlukan modul pemrogram tambahan. Mikrokontroler ini dirancang untuk menangani komputasi berkecepatan tinggi sekaligus kebutuhan komunikasi data nirkabel dalam arsitektur *Internet of Things*.

Dalam lingkungan pengembangan profesional, ESP32 dapat diprogram menggunakan ESP-IDF (*Espressif IoT Development Framework*) yang berjalan secara *native* di atas sistem operasi waktu nyata FreeRTOS. Berbeda dari siklus program Arduino yang bertumpu pada `setup()` dan `loop()`, program berbasis ESP-IDF beroperasi melalui pembagian tugas (*task*) dengan titik masuk awal berupa fungsi `app_main()`. Penundaan eksekusi waktu ditangani menggunakan fungsi seperti `vTaskDelay()`, sehingga instruksi penundaan tersebut tidak memblokir antrean tugas sistem lainnya.

1.2.4 Konfigurasi Resistor Pull-up dan Pull-down

Pin *General Purpose Input/Output* (GPIO) yang dikonfigurasi sebagai masukan digital memerlukan referensi level tegangan yang pasti ketika sirkuit tombol mekanik berada dalam kondisi terbuka. Tanpa rangkaian pendukung, pin berada dalam kondisi melayang (*floating*) akibat tingginya impedansi masukan gerbang CMOS, sehingga derau elektrostatik di sekitar dapat memicu pembacaan logika acak antara HIGH dan LOW. Untuk mengatasi kondisi ini, diperlukan resistor referensi untuk menstabilkan level tegangan pin.

Konfigurasi *pull-up* menghubungkan pin masukan ke jalur tegangan sumber melalui resistor sehingga bernilai default HIGH saat tombol terbuka, dan beralih ke LOW saat tombol ditekan ke rel arde. Sebaliknya, konfigurasi *pull-down* menghubungkan pin ke jalur arde untuk memastikan logika default LOW saat tidak aktif. ESP32 mendukung penerapan kedua konfigurasi tersebut baik melalui penambahan resistor fisik eksternal sebesar 10 kΩ maupun melalui pengaktifan resistor internal via perangkat lunak, kecuali pada pin masukan khusus (GPIO 34–39) yang tidak memiliki fasilitas penarik internal.

1.2.5 Fenomena Bouncing dan Algoritma Debouncing

Sakelar maupun tombol berbasis kontak fisik memiliki sifat kelenturan mekanis yang menyebabkan permukaannya memantul sesaat ketika ditekan atau dilepaskan. Fenomena pantulan ini (*bouncing*) menghasilkan serangkaian sinyal transisi cepat antara level HIGH dan LOW dalam rentang beberapa milidetik sebelum kontak terhubung sempurna. Mengingat mikrokontroler mampu mengeksekusi

instruksi dalam kecepatan tinggi, osilasi transisi mekanik tersebut dapat disalahartikan sebagai aksi penekanan berulang kali oleh sistem komputasi.

Penanganan fenomena tersebut dapat dilakukan melalui perangkat lunak menggunakan algoritma *debouncing*. Algoritma ini memvalidasi perubahan status input dengan mengukur kestabilan sinyal menggunakan pewaktu sistem, seperti fungsi `millis()`, selama durasi ambang tertentu (antara 20 hingga 50 milidetik). Apabila level logika masukan mampu bertahan stabil melampaui interval waktu tersebut, barulah perubahan status dianggap sebagai masukan yang valid, sehingga pembacaan nilai counter menjadi presisi.

1.2.6 Mode Analog Input dan Pembacaan ADC

Selain difungsikan sebagai keluaran maupun masukan digital biner, pin GPIO mikrokontroler dapat difungsikan dalam mode masukan analog untuk membaca besaran tegangan kontinu. Fungsi ini direalisasikan oleh modul periferal *Analog-to-Digital Converter* (ADC) yang bertugas mengonversi tingkat tegangan fisik menjadi representasi data digital diskret. Pada mikrokontroler ESP32, resolusi ADC bawaan adalah 12-bit, yang memetakan rentang tegangan masukan analog 0–3,3 V menjadi nilai terkuantisasi dari 0 hingga 4095.

Implementasi pembacaan analog dapat diterapkan pada komponen resistif terhadap rangsangan fisik luar, seperti sensor cahaya *Light Dependent Resistor* (LDR). Sensor ini dirangkai secara seri membentuk sirkuit pembagi tegangan bersama resistor referensi untuk menghasilkan perubahan tegangan listrik proporsional terhadap perubahan intensitas foton. Melalui fungsi pembacaan analog seperti `analogRead()`, nilai mentah diskret dapat diakuisisi untuk selanjutnya dikonversi secara matematis menjadi estimasi tegangan nyata guna pemantauan intensitas cahaya secara kuantitatif.

2 Tugas Pendahuluan

Bagian ini berisi jawaban dari tugas pendahuluan yang telah anda kerjakan, beserta penjelasan dari jawaban tersebut

1. Perbedaan mendasar antara *framework* Arduino dan ESP-IDF terletak pada paradigma eksekusi, tingkat abstraksi, serta arsitektur sistem operasi. Framework Arduino mengusung paradigma *super-loop* sekuensial yang disederhanakan melalui fungsi `setup()` dan `loop()`. Sebaliknya, ESP-IDF dibangun secara *native* di atas sistem operasi waktu nyata FreeRTOS dengan struktur berbasis komponen/tugas (*multi-tasking*), di mana eksekusi berawal dari fungsi `app_main()`. Arduino menyembunyikan detail konfigurasi register di balik API tingkat tinggi yang ramah pemula dan memiliki dukungan pustaka komunitas yang sangat luas, namun memiliki *overhead* komputasi serta kontrol perangkat keras yang terbatas. Sementara itu, ESP-IDF menawarkan akses langsung ke tingkat register, efisiensi memori, performa maksimal, serta fitur keamanan industri, meski memiliki kurva pembelajaran yang lebih curam dan struktur kode yang lebih kompleks.

2. Board STM32 Blackpill memerlukan pemrogram eksternal seperti ST-Link karena pada tata letak PCB-nya tidak disematkan cip jembatan USB-ke-Serial (*USB-to-UART converter*). Port USB tipe-C pada Blackpill terhubung langsung ke pin USB OTG mikrokontroler STM32 (PA11 dan PA12) yang secara bawaan belum memiliki protokol *bootloader* serial siap pakai. Oleh sebab

itu, proses pengunggahan program dan *debugging* memerlukan ST-Link yang berkomunikasi langsung dengan inti ARM Cortex-M melalui jalur antarmuka *Serial Wire Debug* (SWD). Sebaliknya, board ESP32 DevKit telah dilengkapi cip USB-ke-Serial terintegrasi (seperti CP2102 atau CH340) beserta sirkuit *auto-reset*, sehingga mikrokontroler dapat langsung berkomunikasi dengan komputer dan mengunduh *firmware* melalui pin serial UART bawaan hanya dengan kabel USB biasa.

3. Kondisi logika (HIGH/LOW) pada kedua konfigurasi resistor tersebut adalah sebagai berikut:

   - **Konfigurasi Pull-up:** Saat tombol **tidak ditekan**, pin GPIO ditarik menuju rel tegangan sumber (V_{CC}) melalui resistor sehingga terbaca berlogika **HIGH** ($3{,}3$ V). Saat tombol **ditekan**, jalur terhubung langsung ke arde (*ground*) sehingga level tegangan jatuh dan terbaca berlogika **LOW** (0 V, berkarakteristik *active-low*).
   - **Konfigurasi Pull-down:** Saat tombol **tidak ditekan**, pin GPIO ditarik menuju rel arde melalui resistor sehingga terbaca berlogika **LOW** (0 V). Saat tombol **ditekan**, jalur terhubung langsung ke rel sumber tegangan (V_{CC}) sehingga terbaca berlogika **HIGH** ($3{,}3$ V, berkarakteristik *active-high*).

4. Fenomena *bouncing* terjadi akibat sifat mekanik elastis pada pelat kontak logam sakelar atau tombol. Ketika tombol ditekan atau dilepas, permukaan logam mengalami getaran atau pentalan mikro fisik selama beberapa milidetik (1 ms $- 50$ ms) sebelum membentuk kontak listrik yang stabil. Karena frekuensi prosesor mikrokontroler sangat tinggi, osilasi transisi sinyal acak antara HIGH dan LOW tersebut terbaca sebagai penekanan tombol berulang kali (*false triggering*). Dua pendekatan untuk mengatasinya adalah:

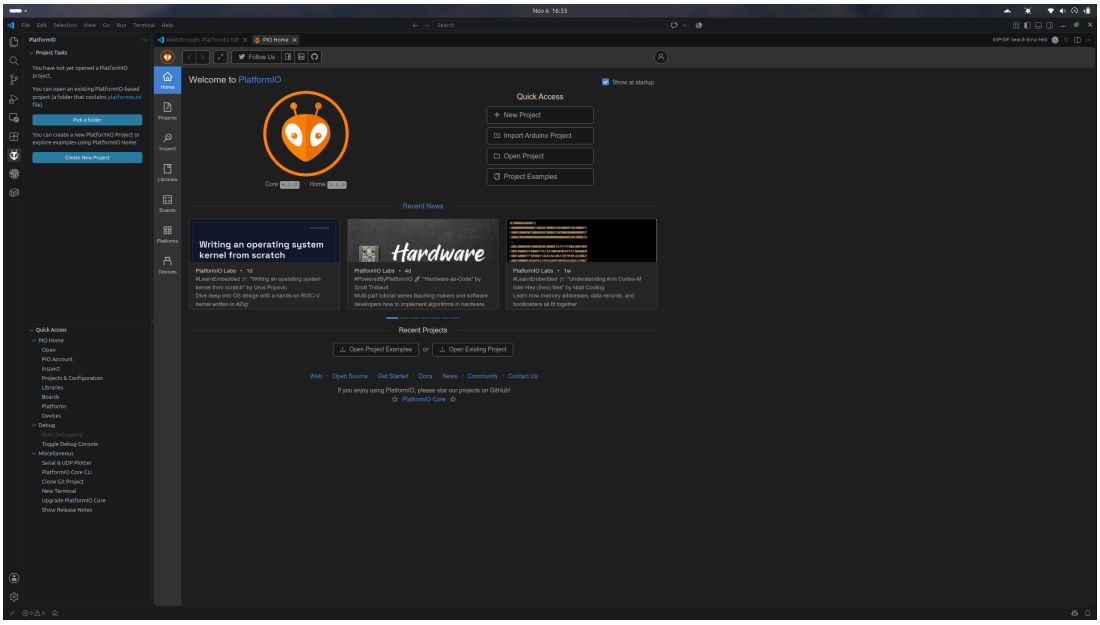
   - ***Software Debouncing*:** Memanfaatkan pewaktu berbasis perangkat lunak (seperti fungsi `millis()`) untuk memverifikasi bahwa perubahan logika masukan telah bertahan stabil melampaui ambang batas waktu tertentu (misalnya 20 ms $- 50$ ms) sebelum dianggap sebagai masukan valid.
   - ***Hardware Debouncing*:** Menambahkan tapis lolos rendah (*low-pass filter*) berbasis rangkaian Resistor-Kapasitor (RC) paralel pada tombol untuk meredam lonjakan tegangan transien, yang sering dipadukan dengan gerbang logika *Schmitt Trigger* guna menghasilkan transisi tegangan logika yang tegas.

5. ADC (*Analog-to-Digital Converter*) mengubah sinyal tegangan analog yang kontinu menjadi representasi digital diskret melalui tiga tahapan: pencuplikan (*sampling*) sinyal secara periodik, penahanan level tegangan (*hold*), serta proses kuantisasi dan pengkodean (*quantizing and encoding*) dengan membandingkan tegangan sampel terhadap tegangan referensi sistem (V_{ref}). Nilai digital keluaran dirumuskan melalui:

$$D_{out} = \left\lfloor \frac{V_{in}}{V_{ref}} \times (2^N - 1) \right\rfloor \tag{1}$$

   Resolusi bit (N) berbanding lurus dengan presisi pembacaan karena menentukan jumlah tingkatan kuantisasi sebesar 2^N. Semakin tinggi nilai resolusi bit, semakin kecil tegangan langkah minimum (*Least Significant Bit* / $\Delta V = \frac{V_{ref}}{2^N - 1}$) yang dapat dideteksi. Hal ini menghasilkan galat kuantisasi (*quantization error*) yang semakin kecil dan kurva representasi sinyal digital yang jauh lebih halus serta mendekati kondisi fisik aslinya.

6. Penyiapan perangkat lunak pengembangan telah diselesaikan pada lingkungan Visual Studio Code dengan instalasi ekstensi PlatformIO IDE serta Serial Plotter.



Gambar 1: Tangkapan layar verifikasi instalasi PlatformIO IDE, platform Espressif 32, platform ST STM32, dan ekstensi Serial Plotter pada VSCode.